



Cpp presentation prepared by	Izaz Ali

Smart Traffic Management System Presentation

Slide 1 — Title Slide

Smart Traffic Management System

Using C++ and Object-Oriented Programming

Developed Using:

- Classes and Objects
- Multithreading

- Sensors Simulation
 - Density-Based Traffic Control
 - Ambulance Priority System
-

Slide 2 — Introduction

What is a Smart Traffic Management System?

A Smart Traffic Management System is an intelligent system that:

- Controls traffic automatically
- Reduces congestion
- Gives priority to emergency vehicles
- Uses traffic density data
- Improves road efficiency

This project simulates a real metropolitan traffic intersection.

Slide 3 — Main Features of the Project

Features

1. Four-lane traffic control
 2. Vehicle density detection
 3. Smart lane selection
 4. Ambulance priority handling
 5. Automatic signal switching
 6. Real-time lane status display
 7. Multithreading support
 8. Random traffic simulation
-

Slide 4 — Libraries Used

Header Files

```
#include <iostream>
```

```
#include <thread>
#include <chrono>
#include <mutex>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>
```

Purpose of Each Library

Library	Purpose
iostream	Input and output
thread	Multithreading support
chrono	Time delays
mutex	Prevent data conflicts
vector	Store multiple lanes
string	Handle lane names
cstdlib	Random number generation
ctime	Random seed using time

Slide 5 — TrafficLight Class

Purpose

This class represents one traffic lane.

Each lane contains:

- Signal state
 - Ambulance detection
 - Vehicle density
 - Lane name
-

Slide 6 — TrafficLight Class Variables

```
string laneName;
bool green;
bool ambulanceDetected;
int vehicleDensity;
```

Explanation

Variable	Function
laneName	Stores lane direction
green	Indicates GREEN or RED light
ambulanceDetected	Detects emergency vehicle
vehicleDensity	Stores traffic density

Slide 7 — Detection System Module

Detection System

Responsible Function

```
void updateDensity()
{
    vehicleDensity = rand() % 101;
}
```

Function

This module simulates traffic sensors.

Working

- Random values between 0–100 are generated
- Represents number of vehicles
- Higher value means heavier traffic

Real-Life Equivalent

- Infrared sensors
 - AI cameras
 - Pressure sensors
 - Radar systems
-

Slide 8 — Ambulance Detection Module

Emergency Detection System

Responsible Function

```
void detectAmbulance(bool state)
```

and

```
void triggerAmbulance(string laneName)
```

Function

This module:

- Detects ambulance presence
- Activates emergency priority
- Forces signal GREEN for emergency lane

Real-Life Technologies

- RFID
- GPS tracking
- Siren recognition AI
- IoT emergency systems

Slide 9 — Smart Decision-Making Module

AI-Based Lane Selection

Responsible Function

```
void followSmartCycle()
```

Working

The system:

1. Checks all lane densities
2. Finds maximum traffic
3. Gives GREEN signal to busiest lane

Code Logic

```
if (lanes[i].vehicleDensity > maxDensity)
```

Benefit

- Reduces congestion
- Improves vehicle flow
- Saves fuel and time

Slide 10 — Signal Control Module

Traffic Signal Management

Responsible Function

```
void setGreenLane(int laneIndex)
```

Working

- One lane becomes GREEN
- All others become RED

Logic

```
lanes[i].setGreen(i == laneIndex);
```

Real-Life Equivalent

Traffic signal controllers at intersections.

Slide 11 — Ambulance Priority Module

Emergency Override System

Responsible Function

```
bool handleAmbulancePriority()
```

Working

- Checks every lane
- Detects ambulance
- Overrides normal traffic cycle
- Gives immediate GREEN signal

Importance

- Saves lives
- Reduces emergency delay
- Provides intelligent response

Slide 12 — Display and Monitoring Module

Monitoring System

Responsible Function

```
void displayAllLanes()
```

and

```
void displayStatus()
```

Function

Displays:

- Lane name
- Signal color
- Density value
- Ambulance alert

Example Output

```
North Lane: GREEN | Density: 80  
South Lane: RED | Density: 45
```

Slide 13 — Multithreading and Synchronization

Thread Safety System

Responsible Object

```
mutex mtx;
```

Function

Prevents:

- Data corruption
- Simultaneous modification
- Signal conflicts

Locking Mechanism

```
mtx.lock();  
mtx.unlock();
```

Importance

Essential in real-time smart systems.

Slide 14 — Timing Control Module

Automatic Delay System

Responsible Function

```
this_thread::sleep_for(chrono::seconds(60));
```

Function

- Waits 60 seconds before next cycle
- Simulates real traffic timing

Real-Life Equivalent

Traffic timer controller.

Slide 15 — Main System Controller

Central Processing Module

Responsible Function

```
void runSystem()
```

Responsibilities

This is the brain of the system.

It:

1. Updates traffic density
 2. Detects ambulance
 3. Makes decisions
 4. Changes signals
 5. Displays results
 6. Repeats continuously
-

Slide 16 — Main Function

Program Starting Point

```
int main()
```

Working

```
vector<string> laneNames = {"North", "South", "East", "West"};
```

Creates four lanes.

```
TrafficSystem system(laneNames);
```

Creates traffic system object.

```
system.runSystem();
```

Starts the complete traffic management system.

Slide 17 — Object-Oriented Concepts Used

OOP Concepts

Concept	Usage
Class	TrafficLight, TrafficSystem
Object	system
Encapsulation	Data and functions together
Abstraction	Hidden internal logic
Modularity	Separate modules

Slide 18 — Real-World Applications

Applications

- Smart cities
 - Metropolitan intersections
 - Highway management
 - Emergency vehicle routing
 - AI traffic systems
 - IoT-based transportation
-

Slide 19 — Advantages of the System

Benefits

- Reduces traffic congestion
- Saves fuel
- Reduces waiting time
- Handles emergencies efficiently

- Improves traffic automation
 - Scalable for smart cities
-

Slide 20 — Future Improvements

Future Enhancements

1. AI camera integration
 2. Real sensor connection
 3. Machine learning prediction
 4. Mobile app monitoring
 5. Cloud-based traffic analytics
 6. Adaptive signal timing
 7. Vehicle counting using computer vision
-

Slide 21 — Conclusion

Conclusion

This Smart Traffic Management System demonstrates:

- Intelligent traffic control
- Density-based decision making
- Emergency vehicle prioritization
- Real-time automation

The project combines:

- C++
- OOP
- Multithreading
- Smart system concepts

to simulate a modern intelligent traffic control solution.

Q1: Why is mutex used?

To avoid simultaneous access conflicts.

Q2: Why use random density?

To simulate real traffic conditions.

Q3: What is the role of multithreading?

To handle concurrent operations efficiently.

Q4: Which function controls smart lane selection?

```
followSmartCycle()
```

Q5: Which module handles emergency vehicles?

```
handleAmbulancePriority()
```

Q6: What type of programming is used?

Object-Oriented Programming.